🚀 Beginner IUPC Preparation: What You Must Know

(A complete learning roadmap & strategy guide for C++ competitive programming)

📘 1. Core C++ Foundations (C থেকে C++ এ উত্তরণ)

যেহেতু তোমরা C ভাষা জানো, C++ এ আসা তোমাদের জন্য খুব কঠিন হবে না। তবে C++ এর কিছু বিল্ট-ইন ফিচার জানা থাকলে কোড লেখার গতি এবং ক্লিননেস অনেক বেড়ে যায়।

key Concepts to Master:

- `<bits/stdc++.h>` **Header:** C-তে যেমন আলাদা আলাদা কাজের জন্য `#include <stdio.h>` বা `<math.h>` দিতে হতো, C++ এ শুধু `#include <bits/stdc++.h>` লিখলেই প্রায় সব স্ট্যান্ডার্ড লাইব্রেরি একসাথে যুক্ত হয়ে যায়।
- **Built-in Helper Functions:**
  - `swap(a, b)`: দুটো ভ্যারিয়েবলের মান অদল-বদল করার জন্য আলাদা থার্ড ভ্যারিয়েবল লাগে না।
  - `max(a, b)` / `min(a, b)`: একাধিক মানের মধ্যে বড় বা ছোট মান এক লাইনে বের করে ফেলা যায়।
  - `abs(x)`: কোনো সংখ্যার অ্যাবসিওলিউট বা ধনাত্মক (positive) মান নেওয়ার জন্য।
- **Passing by Reference (`&`):** ফাংশনে ভ্যারিয়েবল পাঠানোর সময় কপির বদলে মেইন মানকে আপডেট করতে `&` ব্যবহার করা শেখা উচিত (যেমন: `void update(int &x)`)।

🎁 2. Essential C++ STL (Standard Template Library)

অ্যালগরিদম বা প্রবলেম সলভিংয়ে STL হচ্ছে সবচেয়ে বড় হাতিয়ার। C-তে যেই ডাটা স্ট্রাকচারগুলো বানাতে ৫০ লাইন কোড লাগতো, C++ STL দিয়ে তা ১-২ লাইনে করা সম্ভব।

◆ Vector (Dynamic Array)

সাধারণ অ্যারের সাইজ ফিক্সড (যেমন: `int arr[100]`), কিন্তু `vector` প্রয়োজন অনুযায়ী বড় বা ছোট হয়।

- **কী কী জানতে হবে:**
  - `push_back()` দিয়ে উপাদান যোগ করা এবং `pop_back()` দিয়ে শেষ উপাদান মোছা।
  - `v.size()` দিয়ে অ্যারেতে কয়টি এলিমেন্ট আছে তা জানা।
  - `sort(v.begin(), v.end())` ব্যবহার করে $O(N \log N)$ এ অ্যারে সর্ট করা।

- `reverse(v.begin(), v.end())` দিয়ে উপাদানগুলো উল্টানো।
- **Range-based Loop:** `for (int x : v)` দিয়ে সহজে পুরো ভেক্টর ঘোরানো।

◆ Pair

দুটি আলাদা ডাটা টাইপকে (যেমন: একটা `int` আর একটা `string`) জোড়া বেঁধে রাখার জন্য `pair` ব্যবহার করা হয়।

- **কী কী জানতে হবে:**
- `pair<int, int> p` অথবা `vector<pair<int, int>> v` ব্যবহার করা।
- `p.first` এবং `p.second` দিয়ে মান অ্যাক্সেস করা।
- কোঅর্ডিনেট (x, y) বা নোডের সাথে ওয়েট (node, weight) ট্র্যাকিংয়ের ক্ষেত্রে এটি প্রচুর লাগে।

◆ Set & Multiset (Automatic Sorting Data Structures)

- **Set:** সবসময় **অনন্য (unique)** এলিমেন্ট রাখে এবং স্বয়ংক্রিয়ভাবে **ছোট থেকে বড় (sorted)** সাজিয়ে রাখে।
- `s.insert(x)` দিয়ে মান ঢোকানো, `s.erase(x)` দিয়ে মোছা।
- `s.count(x)` বা `s.find(x)` দিয়ে কোনো এলিমেন্ট আছে কিনা তা $O(\log N)$ এ খুঁজে বের করা।
- **Multiset:** `set`-এর মতোই সব সর্টেড রাখে, তবে এতে **ডুপ্লিকেট (duplicate)** মান রাখা যায়।

◆ Map (Frequency Counting & Associative Array)

অ্যারেতে আমরা ইনডেক্স হিসেবে শুধু সংখ্যা ব্যবহার করতে পারি, কিন্তু `map`-এ যেকোনো কিছু (যেমন: string) ইনডেক্স হিসেবে ব্যবহার করা যায়।

- **কী কী জানতে হবে:**
- `map<string, int> freq;`
- কোনো শব্দ বা সংখ্যা কতবার আসলো তা ট্র্যাক রাখা (`freq["apple"]++`)।
- `map` দিয়ে প্রবলেমের ফ্রিকোয়েন্সি হিসাব করা অত্যন্ত সহজ হয়ে যায়।

🔥 3. Time Complexity & Integer Overflow (খুব গুরুত্বপূর্ণ!)

কনটেস্টে প্রবলেম সলভ করার পরও **TLE** (Time Limit Exceeded) বা **WA** (Wrong Answer) আসার মূল কারণ এই দুটি কনসেপ্ট না বোঝা।

◆ Time Complexity Concept

- সাধারণত ১ সেকেন্ডে কম্পিউটার 10^8 টি অপারেশন চালাতে পারে।
- **Complexity Rules:**

- $O(1)$: ইনপুট যত বড়ই হোক, সময় একই লাগবে।
- $O(N)$: $N = 10^8$ হলে ১ সেকেন্ডের মধ্যে চলবে (সিঙ্গেল লুপ)।
- $O(N \log N)$: $N = 10^6$ পর্যন্ত চলবে (যেমন: `sort()` ফাংশন)।
- $O(N^2)$: $N = 10^4$ এর বেশি হলে TLE খাবে (যেমন: নেস্টেড `for` লুপ)।
- প্রশ্ন দেখার সাথে সাথে ইনপুট সাইজ (N) দেখে সিদ্ধান্ত নিতে হবে কোডের কমপ্লেক্সিটি কেমন হতে পারবে।

◆ Integer Overflow & Boundary Values

- C++ এ সাধারণ `int`-এর লিমিট $\approx -2 \times 10^9$ থেকে $+2 \times 10^9$।
- যদি দুটো 10^5 সাইজের সংখ্যা গুণ করা হয় ($10^5 \times 10^5 = 10^{10}$), তবে তা `int`-এর সীমা ছাড়িয়ে ভুল মান (Garbage Value) দেবে।
- **সমাধান:** বড় হিসাবের ক্ষেত্রে অবশ্যই `long long int` ব্যবহার করতে হবে।

🔤 4. Strings & Essential Math Concepts

◆ String Handling

- C-এর মতো `char arr[]` ব্যবহার না করে C++ এর `string` ব্যবহার করা শিখতে হবে।
- **প্রয়োজনীয় টপিক:**
  - String Concatenation (যেমন: `s3 = s1 + s2`)।
  - `reverse(s.begin(), s.end())` দিয়ে পালিন্ড্রোম (Palindrome) চেক করা।
  - `toupper()` এবং `tolower()` দিয়ে ক্যারেক্টার কেস চেঞ্জ করা।
  - অক্ষর বা ক্যারেক্টারের ফ্রিকোয়েন্সি বের করা।

◆ Math for Beginners

- **Prime Number Check:** ১ থেকে N পর্যন্ত লুপ না চালিয়ে, $\sqrt{N}$ পর্যন্ত লুপ চালিয়ে প্রাইম নম্বর চেক করার $O(\sqrt{N})$ নিয়ম।
- **Divisors of a Number:** কোনো সংখ্যার সব ভাজক বা Divisors বের করার লজিক।
- **GCD & LCM:**
  - C++ বিল্ট-ইন GCD: `__gcd(a, b)`
  - LCM বের করার সূত্র: $\text{LCM}(a, b) = \frac{a \times b}{\text{GCD}(a, b)}$ (তবে $a \times b$ করার সময় যেন overflow না হয়, সেদিকে খেয়াল রাখা)।
- **Modulo Operations:** প্রশ্নের শেষে "Print answer modulo 10^9+7" থাকলে কীভাবে বড় সংখ্যার মোডুলাস করতে হয় তার বেসিক ধারণা।

□ 5. Implementation & Ad-hoc Problem Solving

বিগিনার কনটেস্টে অন্তত ১-২টি প্রবলেম থাকে যেগুলো কোনো কঠিন অ্যালগোরিদম দাবি করে না, কেবল সমস্যাটি বুঝে সঠিকভাবে কোড করার দক্ষতা চায়।

Key Focus Areas:

- **Simulation:** প্রশ্নের নির্দেশনা অনুযায়ী ধাপে ধাপে কোড করে যাওয়া (যেমন: একটা গেমের রুলস কোডে রূপান্তর করা)।
- **Pattern Finding:** লুপ ব্যবহার করে কোনো নির্দিষ্ট প্যাটার্ন তৈরি বা আউটপুট দেওয়া।
- **Corner Cases:** $N=1$ হলে কী হবে? ইনপুট Negative হলে কী হবে? Array-এর শেষ এলিমেন্ট বা সীমানার মানগুলো কাজ করছে কি না—এগুলো চিন্তা করতে শেখা।

🏁 6. Contest Strategy & Team Coordination

IUPC একটি টিম কনটেস্ট। এখানে শুধু ভালো কোডিং জানা যথেষ্ট নয়, স্ট্র্যাটেজি ঠিক না থাকলে জানা প্রবলেমও মিস হয়ে যায়।

1. **Rule 1 (First 10 Minutes):** কনটেস্ট শুরু হতেই কোড করা শুরু করবে না। প্রথম ১০ মিনিট পুরো টিম মিলে **সবগুলো প্রবলেম রিড করবে**।
2. **Rule 2 (Target the Easiest):** সবাই মিলে আলোচনা করে সবচেয়ে সহজ প্রবলেমটি চিহ্নিত করবে এবং সেটি আগে সলভ করবে।
3. **Rule 3 (20-Minute Rule):** কোনো প্রবলেমে টানা **২০ মিনিটের বেশি আটকে থাকবে না**। আইডিয়া মাথায় না আসলে বা বাগ না মিললে সাময়িকভাবে প্রবলেম চেঞ্জ করবে।
4. **Rule 4 (Sample & Corner Cases Check):** কোড সাবমিট করার আগে দেওয়া Sample Input ছাড়াও নিজেরা কিছু **Corner Cases** (যেমন: Minimum input, Maximum input) দিয়ে অন্তত **২ বার Dry Run** করে চেক করবে।
5. **Rule 5 (WA Handling):** Wrong Answer (WA) আসলে প্যানিক করা যাবে না। কোড বারবার সাবমিট না করে পেন-পেপার নিয়ে খাতা-কলমে ট্রেস করবে কোথায় ভুল হচ্ছে।
6. **Rule 6 (Clean Code & Communication):** সিঙ্ক্রোনাইজড ভ্যারিয়েবল নেম ব্যবহার করবে, যেন এক টিমে থাকা অবস্থায় একে অপরের কোড দেখে সহজে বুঝতে পারে।

এই টপিকগুলো তোমরা এক এক করে কভার করলে এবং প্র্যাকটিস করলে খুব চমৎকার একটা কনটেস্ট পারফর্মেন্স দিতে পারবে! All the best!